

iCoRE: An Iterative Correlation-Aware Retriever for Bug Reproduction Test Generation

JUNYI WANG, Zhejiang University, China

JIALUN CAO, The Hong Kong University of Science and Technology, China

ZHONGXIN LIU*, Zhejiang University, China

Automatically generating bug reproduction tests (BRT) from issue descriptions is crucial for facilitating software maintenance. Large Language Model (LLM)-based approaches have shown great potential for this task. Their effectiveness heavily relies on retrieving high-quality context from the codebase. The retrieval phase of existing approaches relies on either traditional methods like BM25 or modern LLM-driven strategies. The LLM-based retrieval strategies typically involve equipping an LLM with tools to autonomously explore the code repository or having it select the most relevant files and code snippets from a provided list as context. However, these retrieval methods suffer from three key limitations: (1) They often employ a unified strategy for retrieving both source code and test cases, overlooking their distinct retrieval requirements. (2) They focus solely on semantic similarity, ignoring function call relationships that reflect behavioral relevance, which often leads to the retrieval of irrelevant context. (3) The retrieval lacks a feedback loop from the generation phase, preventing it from refining the context based on execution results. These limitations collectively result in low-quality context, thereby hindering the accuracy of bug reproduction.

To address these challenges, we propose iCoRE, an iterative, correlation-aware context retrieval approach. iCoRE is explicitly designed to be aware of three key correlations: 1) the correlation between source code and test cases, which requires differentiated retrieval, 2) the correlation between textual semantics and function call structures for accurate relevance assessment, and 3) the correlation between the retrieval and generation phases, which enables iterative feedback and refinement. To evaluate iCoRE, we integrate it with an LLM-based BRT generator and conduct a comprehensive evaluation on the SWT-bench Lite and TDD-bench Verified benchmarks. Experimental results show that our method achieves a Fail-to-Pass rate of 42.0% and 52.8% respectively, representing significant 19.7%–31.7% relative improvements over existing retrieval methods.

CCS Concepts: • **Software and its engineering** → *Software testing and debugging*.

Additional Key Words and Phrases: Bug Reproduction, Code Retrieval, Reproduction Test Generation

ACM Reference Format:

Junyi Wang, Jialun Cao, and Zhongxin Liu. 2026. iCoRE: An Iterative Correlation-Aware Retriever for Bug Reproduction Test Generation. *Proc. ACM Softw. Eng.* 3, FSE, Article FSE186 (July 2026), 26 pages. <https://doi.org/10.1145/3808193>

1 Introduction

Bug reproduction refers to the process of reliably triggering the faulty behavior of a reported software issue, enabling developers to observe, analyze, and ultimately diagnose the underlying

*Corresponding author. Also with Hangzhou High-Tech Zone (Binjiang) Institute of Blockchain and Data Security.

Authors' Contact Information: [Junyi Wang](mailto:wangjunyi@zju.edu.cn), wangjunyi@zju.edu.cn, College of Computer Science and Technology, The State Key Laboratory of Blockchain and Data Security, Zhejiang University, Hangzhou, China; [Jialun Cao](mailto:jialuncao@ust.hk), jialuncao@ust.hk, The Hong Kong University of Science and Technology, Hong Kong, China; [Zhongxin Liu](mailto:liu_zx@zju.edu.cn), liu_zx@zju.edu.cn, College of Computer Science and Technology, The State Key Laboratory of Blockchain and Data Security, Zhejiang University, Hangzhou, China.



This work is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License.

© 2026 Copyright held by the owner/author(s).

ACM 2994-970X/2026/7-ARTFSE186

<https://doi.org/10.1145/3808193>

defect. In a typical software development workflow, once a bug is reported, a developer's task is to first reproduce the bug, then locate the root cause of the problem, and finally write a fix patch to resolve it [29]. The executable test created during this process to reproduce the bug is known as a Bug Reproduction Test (BRT). Given an issue description and its associated buggy code repository, a valid BRT should satisfy two core conditions: (1) it should fail on the buggy version of the code, thereby manifesting the undesired behavior; (2) it should succeed after the fix patch is applied, thereby validating the correctness of the applied fix [11, 23].

Well-constructed Bug Reproduction Tests (BRTs) are crucial for software maintenance, as they help developers locate issues, validate fixes, and prevent regressions [12, 19, 29]. However, manually creating these tests is a time-consuming and labor-intensive process [9, 16, 21]. To address this, the automated generation of BRTs has become a significant area of research [17, 24, 26], which not only enhances the efficiency of human developers but also improves the performance of automated issue-solving agents [30, 32, 34]. For human developers, this automation significantly reduces the burden of manual bug reproduction, allowing them to focus their efforts directly on root cause analysis and implementing fixes [5, 11, 31]. For automated issue-solving agents, many of them incorporate BRT generation as a key component, relying on the resulting tests to validate whether their proposed fixes have correctly resolved the bug [8, 33, 36, 38].

Traditional methods often rely on analyzing specific program crash data like stack traces [10, 13], thus are ill-suited for bugs described in natural language. Recent advances in Large Language Models (LLMs) have established the new state-of-the-art in automated BRT generation, thanks to their strong capabilities in natural language understanding and code generation. Existing LLM-based methods can be divided into two types. Some methods [17, 23] operate by directly prompting an LLM with few-shot examples or context retrieved via BM25. Other methods [1, 2, 18, 31] adopt a more complex, multi-step process: they first employ an LLM-based retriever to localize relevant files and code snippets, and then initiate an iterative cycle of generation, execution, and refinement.

Accurately retrieving relevant context is crucial for the success of LLM-based bug reproduction. Issue descriptions frequently lack the key details required for reproducing bugs [6, 14, 25]. Without sufficient context, an LLM is prone to making errors for bug reproduction [20]. For example, it may use the wrong testing framework or misuse function APIs. In contrast, accurate context equips the model with key information like correct API usage and example test cases, significantly reducing hallucinations and improving generation quality [35, 39, 40]. This necessity is also confirmed by recent studies. For instance, removing context localization causes a 17.7% performance drop for the LLM-based BRT generation approach Otter [1], and for its state-of-the-art successor e-Otter, the success rate was 42.6% among instances with correct test localization, dropping sharply to 24.2% for those with incorrect localization [2].

Existing LLM-based bug reproduction approaches have adopted a variety of retrieval strategies to provide contextual information. Some methods rely on traditional term-based retrieval like BM25 [23], while more recent approaches use Large Language Models to identify relevant context. This is typically done either by equipping the LLM with tools as a search agent to explore the code repository autonomously [31], or by having it perform a hierarchical selection, i.e., first choosing relevant files from a list, and then identifying relevant functions within them [1, 2, 18]. However, these retrievers face the following key limitations:

Absence of Differentiation Between Production Code and Test Code Retrieval. Effective bug reproduction requires two distinct types of context: production code and existing tests, each with unique retrieval needs. Production code, which clarifies API usage, is typically found by matching explicit entities from an issue description, such as function names. In contrast, existing tests, which provide vital structural templates, require understanding the described behavioral scenario. However, the retrieval phase of existing approaches often employs a single, unified

strategy for both, ignoring their distinct requirements. For example, AssertFlip [18] retrieves only potentially faulty production code and completely ignores the vital test templates offered by existing tests. Ultimately, this absence of a differentiated strategy results in an incomplete context, thereby hindering the successful reproduction of bugs.

Neglect of Function Call Relationships. In this work, we define Function Call Relationships as the structural connections between code entities, such as functions, methods, and classes, formed by caller-callee interactions. Both traditional BM25 and modern LLM-driven retrievers ignore this crucial structural information. BM25 relies on term matching, meaning it can only identify shared keywords between a issue description and a piece of code. When LLM-driven strategies analyze a file or code snippet, it determines relevance by understanding the textual meaning and its similarity to the bug description. Consequently, both approaches are prone to retrieving code and tests that are textually or semantically aligned with the issue description but behaviorally irrelevant. For example, as illustrated in Figure 2b, the LLM-based retrieval module of Otter [1] retrieves test cases based on semantic similarity while neglecting function call relationships. Consequently, it retrieves the test case “test_combine_negated_boolean_expression”, whose implementation does not invoke the key functions, is irrelevant, and did not help to reproduce the bug.

Lack of Feedback from Generation to Retrieval. Existing BRT generation approaches treat retrieval as an isolated step performed before generation. This “one-way” workflow completely overlooks the valuable information from the generation phase. By leveraging feedback from the generation phase, such as the textual content and function call structure of the generated test, the retrieval process can be refined to identify more relevant context, thereby enhancing bug reproduction performance. For instance, as illustrated in Figure 2c, an agent might find a suboptimal test like “test_filtering_on_annotate_that_uses_q” and stop. However, the test generated from this context provides valuable feedback that could guide a subsequent retrieval round to the initially missed but helpful context “test_empty_expression_annotation”.

Problem Description

ExpressionWrapper for ~Q(pk__in=[]) crashes.
 I'm reducing some Q objects (similar to what is described in ticket:32554. Everything is fine for the case where the result is ExpressionWrapper(Q(pk__in=[])). However, when I reduce to ExpressionWrapper(~Q(pk__in=[])) the query breaks.

Symptoms

Working for ExpressionWrapper(Q(pk__in=[]))

```
print(queryset.annotate(foo=ExpressionWrapper(Q(pk__in=[]), output_field=BooleanField()).values("foo").query)
SELECT 0 AS "foo" FROM "table"
```

Not working for ExpressionWrapper(~Q(pk__in=[]))

```
print(queryset.annotate(foo=ExpressionWrapper(~Q(pk__in=[]), output_field=BooleanField()).values("foo").query)
SELECT AS "foo" FROM "table"
```

(a) The problem description of “django__django-15213.”

The Highly Relevant Test

```
1 def test_empty_expression_annotation(self):
2     books = Book.objects.annotate(
3         selected=ExpressionWrapper(
4             Q(pk__in=[]),
5             output_field=BooleanField()
6         )
7     )
8     self.assertEqual(len(books), Book.objects.count())
9     self.assertTrue(all(not book.selected for book in books))
```

The Correct BRT: Achievable with iCoRe

```
1 def test_negated_empty_expression_annotation(self):
2     books = Book.objects.annotate(
3         selected=ExpressionWrapper(
4             ~Q(pk__in=[]),
5             output_field=BooleanField()
6         )
7     )
8     self.assertEqual(len(books), Book.objects.count())
9     self.assertTrue(all(not book.selected for book in books))
```

(b) An straightforward method for reproducing “django__django-15213.”

Fig. 1. Motivating Example

To address the above limitations, we propose a novel **iterative Correlation-aware Retrieval** approach, iCoRE, for automated BRT generation. For the limitation of the unified retrieval strategy,

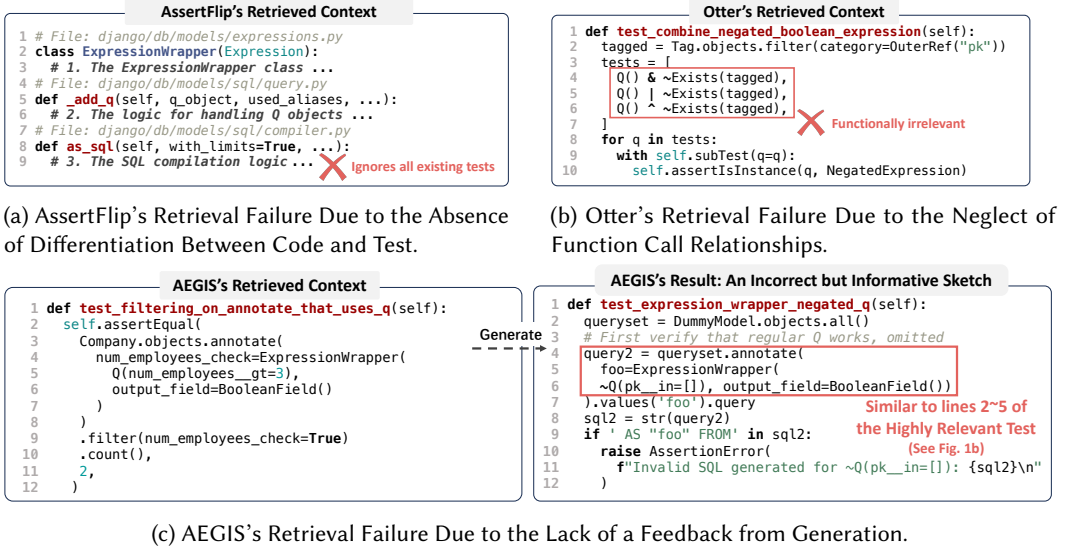


Fig. 2. Motivating Example (Continued from Figure 1)

iCoRE adopts a two-stage process that handles production code and test separately. For production code retrieval, it queries a project-level code index based on keywords in the issue description and heuristic rules. For test code retrieval, we introduce an iterative refinement loop. Within each iteration, we apply a generation module to produce a sketch BRT, which serves as feedback to the retrieval. The sketch is then compared against the existing test suite based not only on textual similarity but also on its function call patterns, which addresses the second and third limitations. iCoRE continuously gains feedback from the generation phase to refine its context, progressively focusing on the most relevant tests.

To evaluate the effectiveness of iCoRE, we connect it to a BRT generator, which utilizes the high-quality context from iCoRE to prompt an LLM for generating BRTs. Our retriever plus this module achieved Fail-to-Pass rates of 42.0% and 52.8% using GPT-4o on SWT-bench Lite and TDD-bench Verified, respectively, representing relative improvements of 19.7%–31.7% over the best prior retrieval methods. Notably, iCoRE is generator-agnostic, meaning it can be readily plugged into various generation modules to enhance the overall performance of BRT generation. Furthermore, an ablation study demonstrated that every component of our framework is indispensable.

Our contributions can be summarized as follows:

- We propose a novel, two-stage iterative context retrieval method for bug reproduction test generation. Specifically, our method: 1) Employs a specialized, two-stage approach for production code and test code; 2) moves beyond simple textual similarity to capture function call relationships for deeper structural relevance; and 3) introduces a feedback loop from generation to retrieval using generated sketch BRTs, enabling iterative context refinement.
- We apply our retrieval method to enhance LLM-based test generation for bug reproduction. We show that providing iteratively refined, high-quality context significantly improves the accuracy and validity of the generated tests.
- We demonstrate the effectiveness of our approach through extensive experiments. On SWT-bench Lite and TDD-bench Verified, our framework achieves Fail-to-Pass rates of 42.0% and 52.8% respectively, validating the superiority of our proposed method.

2 Motivation

In this section, we illustrate the limitations of existing retrieval approaches through a real-world bug, “django__django-15213”, from the SWT-bench benchmark[23]. As shown in Figure 1a, this bug occurs when the system processes a negated empty condition object `Q(pk__in=[])`, which produces invalid SQL and causes a crash. The goal is to generate a test that reproduces this specific error automatically. For this example, a straightforward solution exists. The solution requires retrieving a highly relevant test already present in the project: “test_empty_expression_annotation”, which provides a near-perfect template. The correct test can then be produced by making a simple modification: adding a tilde symbol before the `Q` object on Line 4. However, most existing methods cannot generate a test that successfully triggers this error. After our analysis, the root causes of failures can be attributed to their retrieval, which supplies insufficient context.

The first limitation of existing retrieval methods is their failure to differentiate between the distinct needs of retrieving production code and test. For example, AssertFlip’s [18] retrieval, which uses a unified retrieval strategy, successfully retrieves several relevant code snippets but completely overlooks the distinct retrieval requirements for test cases, as shown in Figure 2a. It overlooks the need for test examples that demonstrate how to construct the scenario that triggers the bug. Consequently, the generator receives insufficient context and produces an incorrect BRT. In contrast, an approach that distinguishes between production code and tests can retrieve not only the faulty code but also the essential test templates required for bug reproduction.

The second limitation of existing retrieval methods is their reliance on textual semantics while ignoring the function call relationships. For example, Otter’s [1] retrieval employs a two-step workflow, which instructs an LLM to first identify relevant files, then select the most relevant function names from that subset. As shown in Figure 2b, Otter’s retrieval ultimately selects “test_combine_negated_boolean_expression” as the relevant test. This choice appears reasonable, as its name contains semantically aligned concepts like “negated” and “expression”. However, this reliance on ambiguous semantics completely ignores the underlying function call relationships. The logic of the retrieved test centers on the interaction between `Q` and `Exists`, but never invokes the very components required to trigger the bug like `ExpressionWrapper`. Consequently, the selected test is functionally irrelevant and serves only as a distraction for LLMs. In contrast, an analysis of function calls confirms that “test_empty_expression_annotation” is behaviorally more relevant, because it invokes the necessary methods for triggering the bug. This highlights the necessity of analyzing function call relationships to determine behavioral relevance.

The third limitation in existing methods is the absence of feedback from generation to retrieval. For example, AEGIS [31] incorporates an LLM-based search agent to perform context retrieval. It located a closer but suboptimal test, “test_filtering_on_annotate_that_uses_q”, as illustrated in Figure 2c, which still diverges from the essence of the bug. Based on this suboptimal context, AEGIS generates a failed BRT. However, this incorrect BRT can serve as a powerful new clue, as it successfully synthesizes the core bug pattern, providing valuable feedback from generation. This feedback can then be used to perform a more targeted search, leading to the highly relevant test, “test_empty_expression_annotation”, that was initially missed. This clearly demonstrates the critical importance of a generation-to-retrieval feedback iteration: by leveraging the sketch BRT from the generation phase as a new, particular query, the initial retrieval errors can be rectified and converge on the better context that static, one-way methods overlook.

The limitations mentioned above motivate us to propose iCoRE, a novel, correlation-aware approach. Specifically, iCoRE is aware of three key correlations: 1) the complementary relationship between production code and test cases, which requires differentiated retrieval; 2) the interplay between textual semantics and function call structures, which must be jointly assessed for behavioral

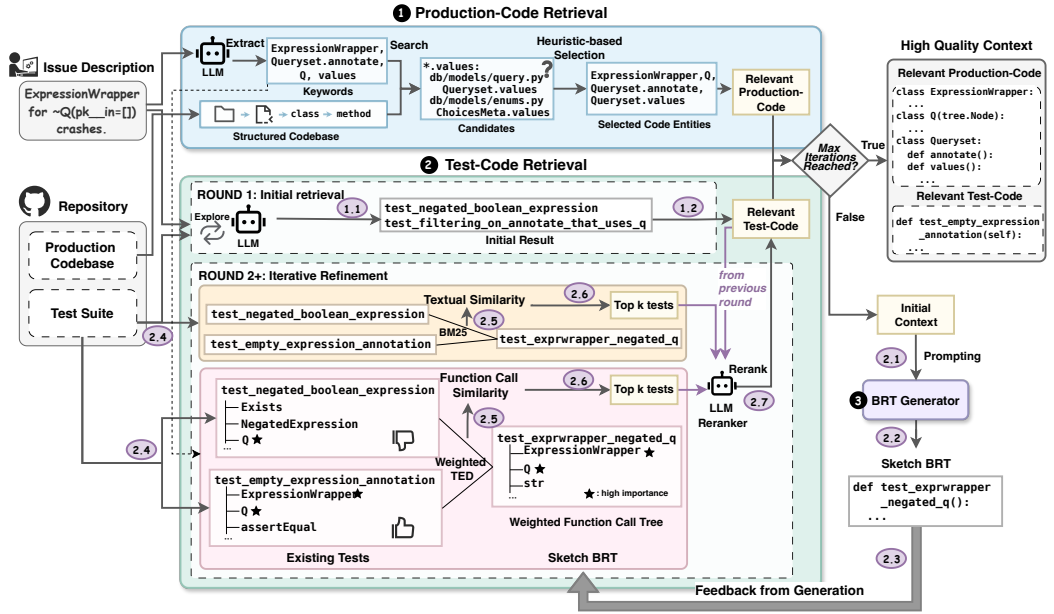


Fig. 3. Overview of iCoRE.

relevance; and 3) the powerful feedback correlation between the generation and retrieval phases, which enables iterative refinement. By being aware of these correlations, iCoRE systematically alleviates the failures of prior approaches.

3 Approach

In this section, we introduce the framework of iCoRE. As illustrated in Figure 3, the overall process of iCoRE takes an issue description and a code repository as input to produce a high-quality context as output. iCoRE consists of three core modules: ① a Production-Code Retrieval module, uses the issue description and the production codebase as input to **retrieve relevant production code**; ② a Test-Code Retrieval module, which takes the issue description and the test suite to **retrieve relevant tests**; and ③ a BRT Generator, an LLM-based framework, generates a sketch BRT based on the issue description and the retrieved context.

The pipeline of iCoRE is as follows: The Production-Code Retrieval module first extracts keywords (i.e., code-related entities) from the issue description. These **keywords** are then searched against a hierarchical structure of the repository's files, classes, and methods to **gather a set of candidates**. Finally, a heuristic-based selection process pinpoints the most relevant code entities to form the relevant production code. The Test-Code Retrieval module executes an iterative process. In the initial round, it utilizes an LLM to explore the test suite based on the issue description and retrieve an initial set of relevant tests. From the second round onwards, the module begins an iterative refinement loop. In each iteration, it leverages the retrieved production code and the test code from the previous round to prompt the BRT Generator for a sketch BRT. This sketch then serves as input to the Test-Code Retrieval module. By calculating textual and function-call similarities between the sketch BRT and existing tests, the module selects a top-k test list for each metric. These tests are then fed into an LLM reranker, which selects and outputs the most relevant tests. This iterative process terminates once the maximum number of iterations is reached. The retrieved code and tests are combined as the context retrieved by iCoRE.

Table 1. Ambiguity in keyword retrieval: Retrieval results for the keywords “annotate” and “values”.

keywords	file	parent
annotate	django/db/models/query.py	Queryset
	django/db/models/query.py	Queryset
values	django/db/models/enums.py	ChoicesMeta
	django/contrib/sessions/backends/base.py	SessionBase

3.1 Production-Code Retrieval

The production-code retriever takes as input an issue description and the project’s production codebase, and outputs a set of relevant production code. The process involves three main steps: keyword extraction, keyword search, and heuristic-based selection.

Keyword Extraction. We utilize LLMs to extract potential code-related entities from the issue description, including function, class, and module names. For example, as shown in Figure 3, we extract keywords such as ExpressionWrapper, Q, annotate, and values. To precisely identify code elements from a issue description, we designed a custom prompt (Appendix B) that guides the model to identify code elements while filtering out irrelevant information. The prompt instructs the model to infer the full entity path (e.g., Queryset.annotate). Recognizing that this inference can be imprecise, if a search for the full path yields no results, we incorporate a fallback strategy that uses only the final entity name (e.g., *.annotate) for the subsequent search.

Keyword searching. We parse the code repository into a hierarchical structure of files, classes, and methods, where each of these elements is treated as a searchable node. By performing string matching between the extracted keywords and the node names in this structural tree, we obtain an initial set of candidates. However, common keywords can appear in multiple locations, such as “values” extracted from the issue description shown in Figure 3. As detailed in Table 1, this results in a high number of irrelevant candidates, which reduces retrieval accuracy.

Heuristic-based Selection. To address this ambiguity, we designed a heuristic-based selection mechanism to identify the most relevant nodes, as detailed in Algorithm 1 (Appendix A). This mechanism considers three key factors: (1) *Node Uniqueness*: Keywords that match only a few nodes are considered more specific and are given higher priority (Algorithm 1, line 7). For instance, in Table 1, “Queryset.annotate” matches a single node, making it a high-confidence result. (2) *File Co-occurrence*: A file is considered more relevant if it contains nodes matched by multiple different keywords (Algorithm 1, line 9). In the example from Table 1, the first candidate for values is in the same file (query.py) as annotate, suggesting they are more relevant. (3) *Structural Relationship*: Nodes that share a parent in the code hierarchy, such as belonging to the same class, are considered more relevant (Algorithm 1, line 17). As seen in Table 1, the parent of the first values candidate is Queryset, same as annotate, indicating a strong structural relationship.

Through this process, the Production-Code retrieval module efficiently filters a large codebase to identify the code snippets most relevant to the issue description, providing high-quality context for the subsequent test generation phase.

3.2 Test-Code Retrieval

The input for this module is the issue description and the project’s test suite. In iterative rounds, it also takes the sketch BRT generated from the previous round. Its output is a set of highly relevant test cases. The retrieval process begins with an initial search and is followed by iterative refinement.

3.2.1 Initial Retrieval. In the first round, we employ an LLM for an exploratory search. To facilitate this exploration, we provide the LLM with a set of tool functions: *list_root* and *list_folder* to list directory structures, *list_tests* to identify test functions within files, and *read_function* to retrieve the production code of specific functions. Guided by the issue description, the LLM identifies an initial set of tests. However, this initial LLM-guided search has inherent limitations. The model may terminate its exploration prematurely once it discovers a test that appears textually relevant and overlook other behaviorally relevant tests that are less textually relevant. Consequently, the initial set of tests is often suboptimal and may not be truly relevant to the bug’s root cause.

3.2.2 Similarity Calculation for Iterative Refinement. To overcome the limitation of the initial search and refine the suboptimal tests, we introduce an iterative process driven by a comprehensive similarity calculation. We initiate this process by first constructing an initial context that combines the initially retrieved tests with the relevant production code from Section 3.1. Based on this context, we generate a sketch BRT (e.g., “test_exprwrapper_negated_q” as shown in Figure 3). Starting from the second round, this sketch BRT is then used as a query to find more relevant tests by calculating its similarity to all existing tests in the test suite. We assess similarity across two dimensions: textual and function call similarity.

For textual similarity, we use the BM25 algorithm to compare the entire code of the generated BRT with existing tests, as well as comparing test function names. We treat test names as a distinct feature because they often concisely summarize a test’s objective and contain keywords highly relevant to the issue description. The final textual similarity score is a combination of the name similarity and the code similarity, calculated as $\text{Similarity}_{\text{textual}} = \text{sim}_{\text{name}} + \text{sim}_{\text{code}}$.

However, relying solely on textual similarity can be misleading. For instance, in the example from Figure 3, the sketch BRT may be textually closer to “test_negated_boolean_expression”. However, “test_empty_expression_annotation” is the most relevant test case in this example. We observe that the function call pattern of the most relevant test cases and the sketch BRTs are often more similar, which motivates our introduction of function call similarity to measure the behavioral resemblance between tests. Specifically, we model these relationships for each test as a pruned Function Call Tree, rooted at the test function itself, and stop expanding the call path once it enters a production module. We chose a pruned tree over a complete call graph for two reasons. First, it allows us to focus on the test’s logic while reducing noise by excluding deep, irrelevant implementation details from the application code. Second, it reduces computational cost, as comparing full project-level call graphs is prohibitive. Furthermore, since test cases are typically independent and do not call one another, this pruning process naturally results in a tree structure rather than a graph.

We employ a Weighted Tree Edit Distance (TED) [28] to quantify the similarity between two call trees. Unlike standard TED, which treats all nodes equally, we consider the different importance of each function. For instance, a function explicitly mentioned in an issue description is far more significant than a common utility function. Thus, we assign a weight $W(f)$ to each node (i.e., function f) in the call tree, which serves as the cost for edit operations (e.g., insertion, deletion).

The weight $W(f)$ for any given function f is determined by the following piecewise function:

$$W(f) = \begin{cases} 1.0 & \text{if } f \in S_{\text{keyword}} \\ \alpha + (0.9 - \alpha) \times \frac{\text{IDF}(f)}{\text{IDF}_{\max}} & \text{otherwise} \end{cases} \quad (1)$$

This formula uses a hybrid strategy, with its components detailed as follows: (1) *Keyword Functions*: Since these function names are directly linked to the bug description, we consider them the strongest indicators of a test’s intent, such as “ExpressionWrapper” and “Q”. The keyword set, S_{keyword} , is directly derived from the code entity names extracted from the issue description by the Relevant Code Retriever in Section 3.1. Accordingly, any function f belonging to this set is assigned a fixed

maximum weight of 1.0. (2) *Non-Keyword Functions*: If it appears in many tests (e.g., “assertEqual”, “str”), it is less distinctive and should receive a lower weight. Conversely, a function that appears in only a few specific tests is more likely to represent the core logic of those tests and should be assigned a higher weight. Therefore, we leverage Inverse Document Frequency (IDF) [27] to assess their importance. The resulting IDF score is then normalized to derive the final weight. The parameter α serves as a base weight to ensure that common functions also have a minimum contribution to the edit cost. The term $(0.9 - \alpha)$ scales the IDF score, capping the weight of non-keyword functions at 0.9 to distinguish them from exact keyword matches (1.0). In this work, we set the default α to 0.1. This configuration ensures that the variable IDF component remains dominant over the base weight and that similarity scores are driven by term specificity.

We apply the classic Zhang-Shasha algorithm [37] for computing the weighted TED (TED_w), between the call tree of the generated test (T_g) and an existing test (T_e), then we normalize this distance into a similarity score using the following formula:

$$\text{Similarity}_{\text{func_call}}(T_g, T_e) = 1 - \frac{TED_w(T_g, T_e)}{W(T_g) + W(T_e)}$$

The denominator is the sum of the total weights of both trees, representing the theoretical maximum edit distance. A tree’s total weight is the sum of all its node weights, $W(T) = \sum_{f \in T} W(f)$. This transforms the distance into a similarity score, where a value closer to 1 indicates a higher degree of similarity in the call structure.

3.2.3 LLM-based Reranking. In the final stage of each retrieval iteration, we use an LLM to synthesize the results and produce a final ranked list of relevant tests. This phase is necessary to filter the large set of potentially noisy candidates from the initial retrieval and to ensure the final selection aligns with the original issue description, not just the sketch BRT. LLM is suited for this task due to its ability to perform analysis across both structured test code and natural language issue descriptions.

The reranker takes three sets of test cases as input: the top-k tests identified by textual similarity, those from function call similarity, and the tests selected in the previous iteration. By providing all three sets, we enable the LLM to act as an expert judge, considering not only the textual and behavioral signals from the current iteration but also the historical context from the previous one. This process allows the reranker to capture complex relationships that a simple score combination might miss, resulting in a more sufficient and helpful context for the next generation stage. While LLMs may make biased judgments, we employ them in the loop for their automation and powerful reasoning capabilities. As our evaluation in Section 5.4 shows, such incorrect judgments are rare.

The process of generating a test sketch, using it to refine retrieving and reranking a new set of tests, concludes one iteration of test retrieval. Upon reaching the maximum number of iterations, the reranker’s final output is taken as the definitive relevant test code. This test code, together with the relevant production code from Section 3.1, constitutes the final high-quality context.

3.3 BRT Generator

This module is responsible for generating the BRT using the high-quality context provided by the retrieval modules. The main part of this module is a carefully designed prompt, the full version of which is detailed in Appendix B. This prompt guides the entire generation process: it first instructs the model to analyze the issue description, explicitly identify the Observed Behavior (OB) and the Expected Behavior (EB). The model is then directed to synthesize this analysis with the provided contextual information. Based on this comprehensive understanding, the model generates the sketch BRT.

4 Experimental Setup

This section systematically evaluates the effectiveness of our proposed approach. We design experiments around three key research questions (RQs):

- RQ1: How effective and efficient is iCoRE in supporting end-to-end bug reproduction compared with different retrieval methods?
- RQ2: How does iCoRE perform when paired with a basic generator, compared to state-of-the-art bug reproduction methods?
- RQ3: What is the contribution of each component of iCoRE to the overall performance?
- RQ4: How accurate is iCoRE at locating target tests compared to baseline retrieval methods and its ablation variants?
- RQ5: How do key hyperparameters impact the performance of iCoRE?

4.1 Dataset

We conduct experiments on two benchmarks, i.e., SWT-bench Lite [22] and TDD-bench Verified [3]. Both of them are widely used in prior work [1, 2, 18, 31]. SWT-bench Lite and TDD-bench Verified, respectively, include 276 and 449 real-world bug instances collected from 12 popular open-source projects. Each instance provides a detailed issue description, the buggy version of the repository, the fix patch, and a ground-truth BRT patch that fails on the buggy version and passes on the fixed version.

4.2 Evaluation Metrics

4.2.1 Bug Reproduction Performance. To evaluate bug reproduction performance in our analyses for RQ1, RQ2, RQ3, and RQ5, we adopt the primary metrics established by the SWT-bench benchmark, which are as follows:

- **Fail-to-Pass Success Rate ($F \rightarrow P(@k)$):** It measures the ability to reproduce a bug successfully. A reproduction is considered successful if a generated test fails on the buggy version, passes on the fixed version, and introduces no new failures on the fixed version ($\times \rightarrow F$). $F \rightarrow P(@k)$ is the percentage of bug instances where at least one of the top k candidates is an $F \rightarrow P$ test.
- **Patch Coverage Score:** This metric measures the extent to which a generated test covers the developer's fix patch. Given the specific implementations of the benchmarks, we adopt the coverage metric provided by each. For SWT-bench Lite, we report Delta Change Coverage (ΔC) [23], calculated as the percentage of modified lines in the patch that are newly covered by the generated test. For TDD-bench Verified, we report Adequacy [3], which measures the ratio of added and deleted lines in the ground-truth patch that are covered by the test execution. In our evaluation, we refer to both metrics collectively as Patch Coverage.

4.2.2 Relevant Test Retrieval Performance. To evaluate the test retrieval module's (detailed in Section 3.2) ability to locate target tests in our analysis for RQ4, we use three standard metrics from the code search domain:

- **Mean Average Precision (MAP):** This metric considers the rank of all relevant items, reflecting the overall quality of the ranking by measuring both precision and recall.
- **Mean Reciprocal Rank (MRR):** This metric measures the rank of the first correctly retrieved item. A higher MRR indicates that a correct item was found earlier in the list.
- **Hit Rate @k (Hit@k):** This metric measures the percentage of queries for which at least one correct item is found within the top k results.

4.3 Comparative Methods

4.3.1 Retrieval Baselines. To compare the capability of iCoRE against existing methods in retrieving context for bug reproduction in our analyses for RQ1 and RQ4, we selected the following diverse set of retrieval baselines:

- **BM25 [23]:** A traditional retrieval method used in its ZeroShot and ZeroShotPlus configurations. We use the official results for this baseline as provided by the SWT-bench.
- **AEGIS's Retriever [31]:** AEGIS is an agent-based BRT generation framework, which employs a search agent that uses tools to explore the repository. While AEGIS itself generates BRT in script mode rather than the unit tests used by our generator and other methods(unit test mode), its retrieval component is suitable for comparison. We directly extract the retrieved context from the official execution logs of AEGIS on the SWT-bench leaderboard.
- **AssertFlip's Retriever [18]:** Employs a hierarchical retrieval method focused exclusively on retrieving production code. It first locates relevant source files and then identifies specific functions within them to serve as context. We use their officially published retrieval results, which are available in their public GitHub repository.
- **Otter's Retriever [1]:** Also uses a hierarchical retrieval approach, but separates the process into two stages to retrieve relevant code and test contexts independently. Since its official retrieval results are not publicly available, we faithfully re-implemented this method based on the paper's description and prompts, using GPT-4o.

4.3.2 BRT Generation Baselines. In our analysis for RQ2, we compare the end-to-end performance of iCoRE paired with a basic generator against several state-of-the-art BRT generation methods reported on the SWT-bench leaderboard, including:

- **ZeroShotPlus[23]:** Proposed by SWT-bench, combining issue descriptions with BM25-retrieved context to directly prompt LLMs for test generation, utilizing a structured diff format to ensure the output is syntactically valid.
- **LIBRO[17]:** Generates multiple candidate tests with few-shot prompting and selects the best one using execution feedback.
- **Amazon Q Developer Agent[4]:** A commercial, closed-source method with unpublished implementation details.
- **AssertFlip [18]:** First generates tests that pass on buggy behavior, then flips assertions to ensure failures on the buggy version.
- **OpenHands[32]:** An open-source AI agent, adapted for bug reproduction, that tackles complex software engineering tasks by creating and executing a detailed plan of action.
- **e-Otter++ [2]:** A feedback-driven method that repairs tests using execution feedback, diversifies candidates with prompting, and leverages generated patches to assist final selection.

4.4 Implementation Details

We utilized three representative LLMs: (1) GPT-4o (gpt-4o-2024-08-06), which is a state-of-the-art closed-source model and is widely used by recent studies in this field, (2) DeepSeek-V3, which is a large-scale open-source model, and (3) Qwen3-32B, which represents a medium-scale open-source model. To ensure deterministic and focused outputs during these steps, the model's temperature was set to 0 for keyword extraction, initial test retrieval, sketch generation, and reranking. In Test-Code Retrieval (Section 3.2), the number of max iterations (mentioned in Section 3.2.3) is set to 3. This means the model generates a sketch BRT and reranks the candidate tests three times in succession. In the reranking stage (Section 3.2.3), LLMs are instructed to select no more than five tests, enforced programmatically. The justification for this parameter choice is explained in RQ5.

Table 2. Bug reproduction performance and context efficiency comparison of different retrieval modules on SWT-bench Lite and TDD-bench Verified. All retrieval methods are paired with the same basic BRT generator. Context efficiency metrics (length and cost) are derived from the GPT-4o experiments.

Benchmark	Retrieval Method	GPT-4o		DeepSeek-V3		Qwen3-32B		Average		Avg. Context Length	Total Input Cost (\$)
		F→P(@1)	F→P(@5)	F→P(@1)	F→P(@5)	F→P(@1)	F→P(@5)	F→P(@1)	F→P(@5)		
SWT-bench Lite	BM25	26.8	31.2	28.3	32.2	31.5	35.5	28.9	33.0	23,562	16.26
	AssertFlip's Retriever*	27.9	34.1	33.3	36.6	30.4	33.7	30.5	34.8	4,324	2.98
	AEGIS's Retriever*	30.8	39.1	33.7	37.7	33.0	37.3	32.5	38.0	3,766	2.60
	Otter's Retriever	31.9	40.2	33.3	35.1	34.4	38.0	33.2	37.8	2,418	1.67
	iCoRe	42.0	46.7	40.2	46.4	41.7	46.7	41.3	46.6	2,603	1.80
TDD-bench Verified	BM25	37.4	41.8	40.1	44.1	37.0	40.8	38.2	42.2	20,882	23.44
	Otter's Retriever	44.1	50.3	44.3	49.2	43.9	49.2	44.1	49.6	2,823	3.17
	iCoRe	52.8	55.2	51.4	55.7	48.1	51.0	50.8	54.0	2,468	2.77

Note: Since the code of AEGIS's retriever and AssertFlip's retriever are not open-source, we utilized their provided retrieval artifacts (GPT-4o) as fixed input contexts when evaluating their performance with DeepSeek-V3 and Qwen3-32B.

To evaluate the end-to-end BRT generation performance(i.e., RQ1, RQ2, RQ3 and RQ5), we applied iCoRe to a basic generator, which is inspired by LIBRO [17], an LLM-based BRT generation framework. Since LIBRO was initially implemented for Java, we adapted it for the Python-based SWT-bench benchmark, involving a four-stage pipeline: prompt-engineering, generation, post-processing, and selection&ranking. The prompt-engineering stage follows the methodology described in Section 3.3, where we integrate the retrieved context into a structured prompt to guide the LLM's generation. Following prior work [17, 18], we perform BRT generation for each sample by creating a new test function and inserting it into an existing test file, regardless of whether the bug requires modifying an existing test or adding a new one. Next, the post-processing stage adapts LIBRO's original Java-based logic for our Python environment, which includes fixing import statements and ensuring test name uniqueness by renaming any conflicting ones. These steps are crucial for producing valid and executable tests. Finally, the selection&ranking stage identifies the best candidate by applying LIBRO's heuristic-based selection approach, which involves clustering candidates by failure type and ranking them based on relevance to the issue description. In the basic BRT generator, we set the model's temperature to 0.7 and generate 10 candidate tests, evaluating the performance on the top-k (k=1, 5) candidates, in line with prior work [17].

5 Experimental Results

5.1 RQ1: Comparison with Retrieval Baselines

To answer RQ1, we evaluate the effectiveness of iCoRe by comparing it against several baselines. We conduct experiments on the two benchmarks i.e., SWT-bench Lite and TDD-bench Verified, with the three models, i.e., GPT-4o, Deepseek-V3, and Qwen3-32B. To isolate the impact of the retrieval component and ensure a fair comparison, each retrieval method is integrated with the same basic BRT generator detailed in 4.4, with the only variable being the retriever itself. To evaluate both the effectiveness and efficiency of each retriever, we report the Fail-to-Pass rates for the top-1 and top-5 selected candidates, as well as the average context length in tokens and the corresponding total input cost based on experiments with GPT-4o. The cost is calculated by summing the individual GPT-4o input costs for each instance across the entire dataset.

The results of our comparison with baseline retrieval methods are presented in Table 2. iCoRe achieves the highest performance across all benchmarks in both metrics. It reaches average F→P(@1) rates of 41.3% on SWT-bench Lite and 50.8% on TDD-bench Verified, outperforming the strongest baseline, Otter's retriever, by 24.4% and 15.2%, respectively. The performance advantage is more pronounced when paired with GPT-4o, where iCoRe outperforms Otter's retriever by 10.1 percentage points (31.7%) and 8.7 percentage points (19.7%) on SWE-bench Lite and TDD-bench

Verified, respectively. In addition, this superiority generalizes to different models: whether paired with DeepSeek-V3 or Qwen3-32B, iCoRE consistently maintains its lead over the baselines on both datasets. These results underscore the effectiveness of iCoRE in identifying more relevant and useful contexts for the BRT generator.

On SWT-bench Lite, the average context length of iCoRE is 2,603 tokens, which is 89.0%, 39.8%, and 30.9% shorter than those of BM25 (over 25K), AssertFlip’s retriever (4,324), and AEGIS’s retriever (3,766). While iCoRE’s context is 6.8% longer than Otter’s on SWT-bench Lite, it delivers vastly superior performance for a marginal increase in cost. Furthermore, on TDD-bench Verified, iCoRE achieves the highest F→P rate (52.8%) with the most concise context (2,468 tokens), which is 88.2% shorter than BM25 (over 20k) and 12.6% shorter than Otter’s retriever (2,823). This result underscores our method’s ability to strike a better balance between context conciseness and completeness, achieving state-of-the-art performance in a highly cost-effective manner.

Answer to RQ1: Experimental results show that iCoRE is more effective and efficient than existing retrieval methods for automated BRT generation. The context provided by iCoRE enables the generation of higher-quality BRTs, achieving a relative improvement of 19.7%–31.7% with GPT-4o over the baseline in F→P(@1) across two benchmarks. This performance advantage generalizes across both closed-source and open-source models. Moreover, iCoRE offers more concise context, averaging ~2,500 tokens, which is 12.6–89.0% shorter than other methods.

5.2 RQ2: Comparison with BRT Baselines

This research question aims to investigate if an advanced context retriever, when combined with a basic generator, can achieve an overall performance that matches or even surpasses that of more structurally complex, state-of-the-art bug reproduction systems. Specifically, we pair iCoRE with the basic generator detailed in 4.4, without any additional feedback or iterative refinements, and evaluate this combination as a self-contained system. We then directly compare its performance with that of several state-of-the-art methods on the SWT-bench Lite leaderboard and TDD-bench Verified. Furthermore, to evaluate the generalizability of iCoRE across different models, we selected not only GPT-4o but also DeepSeek-V3 and Qwen3-32B, as they are representative of the closed-source, open-source, and medium-scale models, respectively.

For a fair and transparent comparison, we adhere to the following data sourcing principles. For SWT-bench Lite, the F→P and Patch Coverage Score(ΔC) metrics for most baselines are taken directly from the official SWT-bench leaderboard. To ensure a fair comparison, we specifically use the result of e-Otter++ with GPT-4o as reported in e-Otter++’s paper [2], since the e-Otter++ entry on the SWT-bench leaderboard [22] uses a different model (i.e., Claude 3.7-sonnet). The ΔC metric for e-Otter++ with GPT-4o was not provided in its paper. For TDD-bench Verified, the baseline results for e-Otter++ are sourced directly from its original paper [2]. The cost for the baseline approaches was sourced from their respective papers. Specifically, these data for OpenHands were calculated from its publicly available logs on the SWT-bench leaderboard. In contrast, these data for the Amazon Q Developer Agent remain unknown due to its closed-source nature. The cost for DeepSeek-V3 and Qwen3-32B is not specified as they are open-source models.

The results in Table 3 demonstrate the superior performance and efficiency of iCoRE. iCoRE with GPT-4o achieves 42.0% and 52.8% in the core metric F→P on SWT-bench Lite and TDD-bench Verified, respectively. It surpasses all existing approaches, including the SOTA method e-Otter++ with GPT-4o (40.2% and 51.4%). In terms of Patch Coverage on SWT-bench Lite(ΔC), our approach achieves 52.4%, which is on par with the record held by the commercial, closed-source tool Amazon Q (52.7%). On TDD-bench, our approach achieves a high Patch Coverage of 76.2%, outperforming

Table 3. Comparison of iCoRE with the Basic Generator against state-of-the-art methods on the SWT-bench Lite and TDD-bench Verified datasets. The Fail-to-Pass rate is measured at $k=1$ ($F \rightarrow P(@1)$). Both $F \rightarrow P$ and Patch Coverage are reported as percentages (%). ‘–’ means unknown and ‘N/A’ means Not Applicable.

Benchmark	Approach	Model	$F \rightarrow P$	Patch Cov.	Cost per Instance (\$)
SWT-bench Lite	ZeroShotPlus	GPT-4	9.4	21.5	0.29
	LIBRO	GPT-4	14.1	23.8	1.52
	OpenHands	Claude 3.5 Sonnet	28.3	52.4	0.35
	AssertFlip	GPT-4o	35.1	44.2	0.96
	Amazon Q	–	37.7	52.7	–
	e-Otter++	GPT-4o	40.2	–	1.80
	iCoRE + Basic Generator	Qwen3-32B	41.7	52.6	N/A
		DeepSeek-V3	40.2	51.7	N/A
		GPT-4o	42.0	52.4	0.28
	e-Otter++	GPT-4o	51.4	70.3	1.80
TDD-bench Verified	iCoRE + Basic Generator	Qwen3-32B	48.1	73.0	N/A
		DeepSeek-V3	51.4	75.4	N/A
		GPT-4o	52.8	76.2	0.26

e-Otter++ (70.3%) by 5.9 percentage points. Collectively, these results demonstrate that our retrieval method, even when paired with a basic generator, can outperform more complex systems on the bug reproduction task.

The average cost per instance of iCoRE + basic generator is merely \$0.26–\$0.28, far below the state-of-the-art approaches. For instance, when using the GPT-4o model, our approach is 6.4 and 3.4 times cheaper than e-Otter++ (\$1.80) and AssertFlip (\$0.96), respectively. This indicates that by providing high-quality context, iCoRE effectively reduces the need for multiple calls to expensive LLMs, achieving high performance through a basic prompt-then-generate process.

Notably, our system (iCoRE + Basic Generator) achieves similar performance using LLMs of different sizes, i.e., success rate 41.7% and 48.1% (Qwen3, 32B) vs. 40.2% and 51.4% (DeepSeek-V3, 671B) on SWT-bench Lite and TDD-bench Verified, respectively. It demonstrates that the powerful capabilities of iCoRE are not limited to leading closed-source models.

Answer to RQ2: Even when paired with a basic generator, the end-to-end performance of iCoRE significantly surpasses current state-of-the-art BRT generation approaches across both SWT-bench Lite and TDD-bench Verified, such as e-Otter++, with GPT-4o. iCoRE offers better cost-effectiveness and is 6.4x cheaper than e-Otter++ when both use the same GPT-4o model. Furthermore, it demonstrates generalizability across different LLMs.

5.3 RQ3: Ablation Study

To measure the contribution of each key component of iCoRE to the BRT generation performance, we conduct a comprehensive ablation study. To achieve this, we create four degraded variants of iCoRE by disabling individual components: ① **w/o Test**: The Test-Code retrieval module is removed. The context consists only of the retrieved production code. ② **w/o Code**: The Production-Code Retrieval module is removed. The context consists only of the retrieved test code. ③ **w/o Iteration**: The iterative refinement loop is disabled. The context includes both the production code and the results from the initial test retrieval. ④ **w/o Function-Call Similarity**: The function call analysis is excluded from the test re-ranking phase. To ensure fairness, the input for the re-ranking process

Table 4. Contribution of each component to the overall bug reproduction performance on SWT-bench Lite and TDD-bench Verified. The Fail-to-Pass rate ($F \rightarrow P(@k)$) is reported as a percentage (%).

Retrieval Component	SWT-bench Lite		TDD-bench Verified	
	$F \rightarrow P(@1)$	$F \rightarrow P(@5)$	$F \rightarrow P(@1)$	$F \rightarrow P(@5)$
iCoRE	42.0	46.7	52.8	55.2
w/o Test	33.7 (↓ 19.8%)	40.6 (↓ 13.1%)	35.2 (↓ 33.3%)	39.9 (↓ 27.7%)
w/o Code	36.6 (↓ 12.9%)	41.7 (↓ 10.7%)	40.8 (↓ 22.7%)	46.8 (↓ 15.2%)
w/o Iteration	37.0 (↓ 11.9%)	41.7 (↓ 10.7%)	44.5 (↓ 15.7%)	49.2 (↓ 10.9%)
w/o Function-Call Similarity	39.5 (↓ 6.0%)	44.6 (↓ 4.5%)	46.5 (↓ 11.9%)	52.1 (↓ 5.6%)

is set to the top 10 textually similar tests and the selected tests. We then evaluate the end-to-end BRT generation of these four variants and compare them against the full iCoRE system.

The results of our ablation study, presented in Table 4, confirm that all of our designed components make positive and significant contributions to the overall performance of iCoRE. Removing test case retrieval (w/o Test) leads to the most severe performance degradation across both benchmarks. On SWT-bench Lite, $F \rightarrow P(@1)$ and $F \rightarrow P(@5)$ rates drop by 19.8% and 13.1%, respectively, while on TDD-bench, the degradation is amplified, reaching 33.3% and 27.7%. Removing code retrieval (w/o Code) also results in a significant drop of $F \rightarrow P(@1/@5)$ by 12.9% and 10.7% on SWT-bench Lite, and by 22.7% and 15.2% on TDD-bench Verified. These results indicate that both test cases and production code play a critical role in BRT generation, but the former provides more critical guidance. This may be because relevant tests not only offer structural templates and assertion logic but also provide function call examples and object instantiation patterns, thereby partially encompassing the functional information found in the production code snippets.

When the iterative process is disabled (w/o Iteration), the $F \rightarrow P(@1)$ performance drops by 11.9% on SWT-bench and 15.7% on TDD-bench. It strongly shows the effectiveness of the feedback loop from a sketch BRT to the retrieval focus, as it enables the system to progressively mine for relevant information that may be missed in a single pass.

Removing the function-call similarity component (w/o Function-Call Similarity) results in a 6.0% and 11.9% $F \rightarrow P(@1)$ performance drop on two benchmarks. It validates our hypothesis that capturing behavioral relevance through function call structures, rather than relying solely on textual matching, enables a more accurate assessment of relevant test cases, thereby further enhancing context quality.

Answer to RQ3: The ablation study confirms the effectiveness of all designs in iCoRE. Both relevant test cases and code snippets play a critical role in the final performance, and test cases make a more significant contribution than production code. The iterative refinement mechanism and the incorporation of function-call similarity also substantially contribute to the overall performance improvement.

5.4 RQ4: Target Test Retrieval Accuracy Evaluation

To evaluate the effectiveness of iCoRE in retrieving target tests, we built a dedicated dataset based on SWT-bench Lite and TDD-bench Verified. We define “target tests” as the test functions explicitly modified by developers during the bug-fixing process. These tests are considered the most directly relevant to the bug, as they are adapted to reproduce its behavior. Accordingly, we extracted these target tests from both datasets to serve as the ground truth for evaluating retrieval accuracy.

We compared the retrieval accuracy of iCoRE against several retrieval baselines that include test retrieval, including BM25, AEGIS’s retriever, and Otter’s retriever. The former two are line-level

Table 5. Comparison of retrieval accuracy for identifying target tests on SWT-bench Lite and TDD-bench Verified. Performance is measured at both file and function levels using MAP, MRR, and Hit@k. ‘-’ indicate that the retrieval artifacts for the corresponding method on the specific benchmark were not available.

Match Type	Method	SWT-bench Lite				TDD-bench Verified			
		MAP	MRR	Hit@1	Hit@5	MAP	MRR	Hit@1	Hit@5
File Match	BM25	0.198	0.210	0.177	0.250	0.139	0.148	0.115	0.198
	AEGIS’s Retriever	0.524	0.552	0.531	0.572	–	–	–	–
	Otter’s Retriever	0.591	0.621	0.596	0.676	0.618	0.655	0.620	0.723
	iCoRE	0.699	0.732	0.684	0.794	0.747	0.786	0.742	0.845
	w/o Iteration	0.561	0.576	0.559	0.596	0.569	0.608	0.592	0.629
	w/o Function Call Similarity	0.685	0.714	0.676	0.765	0.716	0.760	0.728	0.826
Function Match	BM25	0.049	0.051	0.037	0.052	0.012	0.013	0.002	0.018
	AEGIS’s Retriever	0.180	0.214	0.162	0.287	–	–	–	–
	Otter’s Retriever	0.319	0.381	0.346	0.426	0.311	0.370	0.300	0.474
	iCoRE	0.466	0.527	0.471	0.618	0.438	0.507	0.432	0.606
	w/o Iteration	0.344	0.380	0.346	0.434	0.316	0.371	0.310	0.451
	w/o Function Call Similarity	0.449	0.510	0.463	0.588	0.418	0.485	0.413	0.592

retrievers that identify relevant code snippets, whereas the latter uses functions as its unit of retrieval. Additionally, two ablation versions of iCoRE are included in the comparison, i.e., w/o Iteration and w/o Function-Call Similarity.

During the evaluation process, we defined two relevance criteria: ① file-level match, where the retrieved test is in the correct file; and ② function-level match, where either the retrieved test is the ground-truth test itself, or the retrieved line numbers fall within that function’s range. Based on these criteria, we calculated the MAP, MRR, and Hit@k ($k=1, 5$) for both file-level and function-level performance, providing a comprehensive measure of retrieval effectiveness.

Table 5 demonstrates the superior accuracy of iCoRE in retrieving target tests and validates the contributions of its core components. Across all metrics at both the file and function levels, iCoRE significantly surpasses that of all baseline methods and its ablated versions. At the file-match level, iCoRE achieves a MAP of 0.699 on SWT-bench and 0.747 on TDD-bench, outperforming the best-performing baseline, i.e., Otter’s retriever, by 18.3% and 20.9%. Similarly, at the function-match level, iCoRE outperforms the strongest baseline with relative improvements of 46.1% and 36.1% in MAP and Hit@1 on SWT-bench Lite, alongside 40.8% and 44.0% on TDD-bench Verified.

Removing the iteration process (w/o Iteration) causes a drastic drop in accuracy across both benchmarks. The function-level MAP decreases by 26.2% on SWT-bench Lite and 27.9% on TDD-bench Verified. This confirms that the refinement loop is fundamental to discovering relevant contexts. Removing the function-call similarity analysis (w/o Function-Call Similarity) decreases function-level MAP by 3.6% and 4.6% on the two benchmarks, respectively. This indicates that function-call analysis also plays an important role in the accurate retrieval of target tests.

Furthermore, we analyzed whether the LLM reranker (in Section 3.2.3) could cause target tests to be mistakenly overlooked. By comparing the tests fed into the reranker with its final output, we found that only 4% of target tests were filtered out by the LLM. This low error rate confirms that the reranker is effective at preserving relevant tests while improving the overall context quality.

Answer to RQ4: iCoRE is demonstrably more accurate at retrieving relevant tests than both baseline methods and its ablated variants. Its high precision is primarily driven by its iterative refinement process, which is fundamental to its success. This core mechanism is further enhanced by function-call similarity analysis, which provides the necessary fine-grained precision to pinpoint the exact target test function.

5.5 RQ5: Impact of Hyperparameters

We conducted a sensitivity analysis on three key hyperparameters using the SWT-bench Lite dataset: the number of retrieval iterations, the number of relevant tests provided as context, and the base weight used in function similarity calculation.

Impact of Iteration Rounds. We define an iteration as a cycle of generating a sketch BRT and using its feedback to refine the retrieval. Thus, “0 iteration” represents the initial, non-iterative retrieval result, while “k iterations” means k sketch BRTs were generated for feedback. As shown in Figure 4, the performance of iCoRE generally improves with more iterations. Compared to the non-iterative baseline (0 iteration), iterative refinement yields a significant improvement in overall performance. The $F \rightarrow P(@1)$ rate peaks at 42.0% at the third iteration. Although the performance fluctuates slightly in subsequent rounds, with a minor improvement in the $F \rightarrow P(@5)$ rate, the overall performance effectively stabilizes around this peak. This suggests that three feedback cycles are sufficient to gather the critical context for bug reproduction. Therefore, the default setting of three iterations is justified, balancing performance and efficiency.

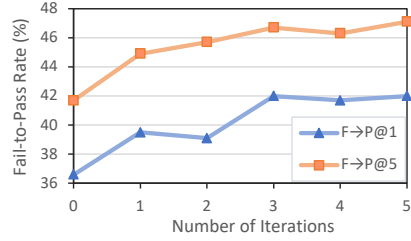


Fig. 4. Impact of Iteration Rounds.

Impact of the Number of Provided Tests. We

investigated the effect of the number of retrieved tests. This is controlled during the reranking stage by limiting the number of tests selected as the final context. We evaluated the performance when providing the top 1, 5, and 10 tests. Table 6 shows that providing five tests yields the best performance on $F \rightarrow P(@1)$. The performance is lowest when only 1 test is provided, because the critical context may not be consistently ranked as the top-1 candidate. When the number is increased to 10, the $F \rightarrow P(@1)$ rate declines by 0.7%, while the $F \rightarrow P(@5)$ rate increases by 0.4%, remaining essentially level with the performance when providing five tests. Consequently, using five tests strikes the optimal balance between providing comprehensive context and maintaining model focus and cost-effectiveness. Therefore, this default setting is reasonable.

Table 6. Impact of the Number of Provided Tests.

# Tests	$F \rightarrow P(@1)$	$F \rightarrow P(@5)$
1	34.4	41.7
5	42.0	46.7
10	41.3	47.1

Impact of Function Similarity Weights. We set the base weight α in Formula 1 (Section 3.2) to be 0.1 by default. To analyze the model’s sensitivity to α , we evaluate iCoRE with the basic generator by varying α from 0.1 to 0.4. This upper bound was selected to ensure that the weight of the IDF component is larger than the base weight, so that the function weight is primarily driven by term specificity. As detailed in Table 7, the $F \rightarrow P(@1)$ rate fluctuates minimally (Mean: 41.3%, Std: 0.45%) with different base weights, demonstrating the robustness of iCoRE to these configurations. In addition, $\alpha = 0.1$ yields optimal performance in our experiments. Therefore, we consider this default setting of α (0.1) to be reasonable.

Table 7. Impact of the base weight parameter α .

Configuration	$F \rightarrow P(@1)$	$F \rightarrow P(@5)$
$\alpha = 0.1$ (Default)	42.0	46.7
$\alpha = 0.2$	40.9	46.3
$\alpha = 0.3$	41.3	46.7
$\alpha = 0.4$	40.9	45.2

Answer to RQ5: iCoRE’s performance is influenced by the number of iterations and the number of provided tests. Conversely, the system demonstrates robustness regarding the function similarity weight (α). Our chosen default settings represent the optimal choice after considering both effectiveness and efficiency.

6 Threats to Validity

Internal Validity. The inherent non-determinism of LLMs poses a potential threat to the reproducibility of our results. To mitigate this threat, we intentionally set the temperature to 0 for all deterministic stages of the retrieval process, including keyword extraction, initial retrieval, sketch generation, and reranking. The second threat relates to the replication of baseline methods. For the Otter’s retrieval baseline, we had to rely on the methodology described in their original papers for replication, as their artifacts were not publicly available. We have strictly followed the descriptions in their papers. The differences between our replication and their actual implementation, though not open-sourced, should be minor.

External Validity. First, our study shares a common threat with other contemporary work [1, 2, 18, 31], i.e., the potential for data contamination [7, 15] within the SWE-bench dataset. The pre-training corpora of the LLMs we used may have contained code or discussions related to the bugs in the benchmark. To assess the impact of this threat, we followed the methodology of Otter [1] and conducted two quantitative analyses on the TDD-bench Verified dataset. We analyzed the performance stability across issue creation years. We focused on the years 2017–2023, excluding earlier years with insufficient sample sizes (<5). Performance across these issue creation years remains relatively stable (Mean: 57.16%, SD: 11.75%), suggesting that the model does not disproportionately favor older, potentially memorized issues. We also measured the textual similarity between our generated tests and existing tests in the repository using the normalized Levenshtein similarity. We found that for 90% of the instances, the similarity score remains below 0.56. These findings confirm that the generator primarily relies on reasoning based on retrieved context rather than memorization, indicating that the impact of data contamination is limited. Second, as our experiments were conducted exclusively on Python projects, our findings may not generalize to other languages. However, Python is one of the most popular languages, and the architecture of iCoRE is language-agnostic and can be adopted to other languages. Finally, our evaluation was performed solely on SWT-bench Lite, which may not fully encompass the diversity of bug types and project complexities encountered in real-world software development. However, this benchmark has been widely used in prior studies [1, 31]. We intend to expand our evaluation to a broader range of programming languages, larger repositories, and more diverse bug scenarios in future work.

7 Discussion

7.1 Comparison with e-Otter++

Regarding e-Otter++ [2], its top performance on the SWT-bench Lite leaderboard (a 50.7% success rate and 56.4% change coverage) was achieved using Claude 3.7-sonnet. However, the cost per instance for e-Otter++ on Claude 3.7 is \$2.75, which is 9.8 times more than our method (\$0.28). Due to these significant cost constraints, we did not compare against it on Claude 3.7. Instead, we chose the widely used commercial model GPT-4o for a fair comparison. Our results show that when using the same GPT-4o model, the context retrieved by iCoRE, even when paired with a basic generator, surpasses the performance of e-Otter++ while incurring only 1/6.4 of the cost, highlighting the superior cost-effectiveness of our iCoRE.

7.2 Failure Analysis

To better understand the limitations of iCoRE, we conducted a qualitative analysis of failure cases of iCoRE paired with GPT-4o and the basic generator on SWT-bench Lite and TDD-bench Verified. We randomly sampled 50 failure instances from each benchmark (90% confidence level, ~10% margin of error) and manually inspected them. The failures were categorized into four primary types:

A. Limitation of Generator (SWT: 58%, TDD: 84%):

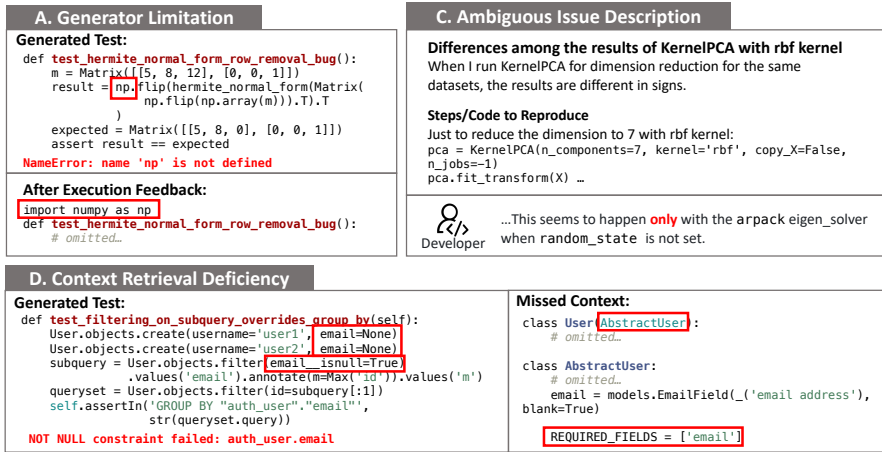


Fig. 5. Examples of Failure Cases

The majority of failures occurred when the retrieved context was sufficient, but the generator failed to output a correct BRT. These failures stem from the limited reasoning capabilities of the model or the simple one-pass generation process. For example, for `sympy__sympy-23413` (Figure 5A), the generated test failed to reproduce the issue simply due to a missing `numpy` import. With an execution feedback loop, the model could fix the `NameError` and successfully reproduce the bug. This underscores the importance of execution feedback in BRT generation. Future work could employ agentic workflows with self-correction capabilities to improve BRT generation.

B. Selection Failure (SWT: 18%, TDD: 10%):

In these cases, the generator produced a correct BRT within the candidate list, but the selection module failed to rank it as the top-1 choice. This suggests that future work should focus on improving the ranking heuristics to better identify the correct test among candidates.

C. Ambiguous Issue Description (SWT: 14%, TDD: 0%):

Some failures stem from the quality of the issue description itself rather than the generator. For instance, for `scikit-learn__scikit-learn-13241` (Figure 5C), the description failed to mention that the bug only occurs when `eigen_solver='arpack'`. Notably, no sample from TDD-bench Verified belongs to this category, as the issue descriptions in TDD-bench Verified are manually verified and filtered for clarity.

D. Context Deficiency (SWT: 10%, TDD: 6%):

In a minority of cases, the retrieval module failed to locate the critical context. For example, for `django__django-11797` (Figure 5D), the retrieval failed to fetch the parent class `AbstractUser`. Lacking this information, the generator failed to recognize that the `email` field has a `NOT NULL` constraint, causing an `IntegrityError` during the test setup. Future work could consider capturing inheritance hierarchies for context retrieval.

8 Related Work

Benchmarks for Bug Reproduction. The systematic evaluation of LLM-based bug reproduction relies on specialized benchmarks. SWT-bench[23], introduced by Mündler et al., is based on real-world Python issues, which evaluates performance through metrics like the Fail-to-Pass (F→P) rate and change coverage. Similarly, Ahmed et al. introduced TDD-Bench-Verified[1], providing a more strictly filtered dataset and introducing a metric that assesses both the F→P property and the test's coverage over code changes, offering a more fine-grained quality measure.

No-Retrieval and Traditional Retrieval Methods. Early LLM-based approaches for generating BRTs are limited by their context retrieval strategies. The no-retrieval approach, LIBRO[17], utilizes a few-shot prompting paradigm, guiding the model with <issue report, test code> examples. Its performance is significantly limited by the complete absence of codebase-specific context in its prompts. To address it, follow-up methods employ traditional information retrieval techniques. For instance, the ZeroShotPlus baseline in SWT-bench[23] uses the BM25 algorithm to retrieve relevant code snippets from the repository. However, such term-frequency-based methods often fail to grasp the semantic meaning of the code, limiting the quality of the retrieved context.

LLM-driven Retrieval Methods. With the advancement of model capabilities, more recent work leverages LLM for more intelligent retrieval. One retrieval method treats the LLM as an autonomous search agent. For example, AEGIS [31] constructs a dual-agent framework that includes a bug-related context summarization module and a finite-state-machine (FSM)-guided script generation module. Another retrieval approach is hierarchical selection, which mimics the developer’s workflow of first locating relevant files and then identifying specific functions within them. AssertFlip [18] adopts this method to locate faulty code for its “pass-then-invert” generation strategy, then treats all context needs as a single bug localization task. The Otter series of methods (Otter, Otter++ [1], e-Otter++ [2]) refines this by using separate stages to retrieve code and test contexts. Otter introduces a “self-reflective action planner” to decide what to read and write, while e-Otter++ further enhances with execution feedback and generated patches. However, it applies the same retrieval strategy to both and relies on function name lists, overlooking complex function call relationships.

Different from existing retrieval methods for BRT generation, iCoRE designs differentiated retrieval strategies for relevant code and test cases, jointly considers textual semantics and function call structures when assessing relevance, and establishes a feedback iteration between retrieval and generation to optimize the context iteratively. By being aware of these correlations, iCoRE can retrieve more accurate and helpful context for bug reproduction, surpassing existing methods. Moreover, our approach is generator-agnostic and can be easily integrated into more advanced BRT generation methods to enhance their overall performance.

9 Conclusion

This paper presents iCoRE, a novel approach for generating high-quality bug reproduction tests by retrieving context using a differentiated, two-stage strategy for code and test case retrieval. By incorporating function call analysis to move beyond simple textual similarity, iCoRE can capture deeper behavioral relevance between tests. Finally, iCoRE established a feedback loop from the generation phase to iteratively refine and improve the quality of the retrieved context. Our comprehensive evaluation on the SWT-bench Lite benchmark demonstrates the superiority of our approach. When integrated with an LLM-based generator, iCoRE achieved a Fail-to-Pass rate of 42.0%, which represents a significant 31.7% relative improvement over the best prior retrieval methods. Moreover, since iCoRE is generator-agnostic, future work can leverage it as a plug-and-play retrieval module, combining it with a more diverse range of generators to explore strategies for further improving the performance of BRT generation.

10 Data Availability

Our replication package, including the code and data, and our online appendix are available at <https://github.com/ZJU-CTAG/iCoRE>.

Acknowledgments

This research/project is supported by the National Natural Science Foundation of China (No.92582107) and the Hong Kong SAR Research Grant Council (General Research Fund Ref: 16206524).

References

- [1] Toufique Ahmed, Jatin Ganhotra, Rangeet Pan, Avraham Shinnar, Saurabh Sinha, and Martin Hirzel. 2025. Otter: Generating Tests from Issues to Validate SWE Patches. In *Proceedings of the 42nd International Conference on Machine Learning*. 752–771.
- [2] Toufique Ahmed, Jatin Ganhotra, Avraham Shinnar, and Martin Hirzel. 2026. Heterogeneous Prompting and Execution Feedback for SWE Issue Test Generation and Selection. In *Proceedings of the 48th International Conference on Software Engineering*. 1262–1273.
- [3] Toufique Ahmed, Martin Hirzel, Rangeet Pan, Avraham Shinnar, and Saurabh Sinha. 2024. TDD-Bench Verified: Can LLMs Generate Tests for Issues Before They Get Resolved? *CoRR* abs/2412.02883 (2024). arXiv:2412.02883
- [4] Amazon Web Services. 2024. Amazon Q. <https://aws.amazon.com/q/>. Accessed: September 1, 2025.
- [5] Moritz Beller, Niels Spruit, Diomidis Spinellis, and Andy Zaidman. 2018. On the Dichotomy of Debugging Behavior Among Programmers. In *Proceedings of the 40th International Conference on Software Engineering*. 572–583.
- [6] Nicolas Bettenburg, Sascha Just, Adrian Schröter, Cathrin Weiss, Rahul Premraj, and Thomas Zimmermann. 2008. What makes a good bug report?. In *Proceedings of the 16th International Symposium on Foundations of Software Engineering*. 308–318.
- [7] Jialun Cao, Wuqi Zhang, and Shing-Chi Cheung. 2024. Concerned with Data Contamination? Assessing Countermeasures in Code Language Model. *CoRR* abs/2403.16898 (2024). arXiv:2403.16898
- [8] Dong Chen, Shaoxin Lin, Muhan Zeng, Daoguang Zan, Jian-Gang Wang, Anton Cheshkov, Jun Sun, Hao Yu, Guoliang Dong, Artem Aliev, Jie Wang, Xiao Cheng, Guangtai Liang, Yuchi Ma, Pan Bian, Tao Xie, and Qianxiang Wang. 2024. CodeR: Issue Resolving with Multi-Agent and Task Graphs. *CoRR* abs/2406.01304 (2024). arXiv:2406.01304
- [9] Mouxiang Chen, Zhongxin Liu, He Tao, Yusu Hong, David Lo, Xin Xia, and Jianling Sun. 2024. B4: Towards optimal assessment of plausible code solutions with plausible tests. In *Proceedings of the 39th International Conference on Automated Software Engineering*. 1693–1705.
- [10] Ning Chen and Sunghun Kim. 2015. STAR: Stack Trace Based Automatic Crash Reproduction via Symbolic Execution. *IEEE Transactions on Software Engineering* 41, 2 (2015), 198–220.
- [11] Runxiang Cheng, Michele Tufano, Jürgen Cito, José Cambronero, Pat Rondon, Renyao Wei, Aaron Sun, and Satish Chandra. 2025. Agentic Bug Reproduction for Effective Automated Program Repair at Google. *CoRR* abs/2502.01821 (2025). arXiv:2502.01821
- [12] Ermira Daka and Gordon Fraser. 2014. A Survey on Unit Testing Practices and Problems. In *Proceedings of the 25th International Symposium on Software Reliability Engineering*. 201–211.
- [13] Pouria Derakhshanfar, Xavier Devroey, Annibale Panichella, Andy Zaidman, and Arie van Deursen. 2020. Botsing, a Search-based Crash Reproduction Framework for Java. In *Proceedings of the 35th International Conference on Automated Software Engineering*. 1278–1282.
- [14] Shihan Dou, Haoxiang Jia, Shenxi Wu, Huiyuan Zheng, Muling Wu, Yunbo Tao, Ming Zhang, Mingxu Chai, Jessica Fan, Zhiheng Xi, et al. 2026. What is wrong with your code generated by large language models? An extensive study. *Science China Information Sciences* 69, 1 (2026), 112107.
- [15] Shahriar Golchin and Mihai Surdeanu. 2023. Time Travel in LLMs: Tracing Data Contamination in Large Language Models. *CoRR* abs/2308.08493 (2023). arXiv:2308.08493
- [16] Roman Haas, Daniel Elsner, Elmar Juergens, Alexander Pretschner, and Sven Apel. 2021. How can manual testing processes be optimized? developer survey, optimization guidelines, and case studies. In *Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 1281–1291.
- [17] Sungmin Kang, Juyeon Yoon, and Shin Yoo. 2023. Large Language Models are Few-Shot Testers: Exploring LLM-Based General Bug Reproduction. In *Proceedings of the 45th International Conference on Software Engineering*. 2312–2323.
- [18] Lara Khatib, Noble Saji Mathews, and Meiyappan Nagappan. 2026. AssertFlip: Reproducing Bugs via Inversion of LLM-Generated Passing Tests. In *Proceedings of the 48th International Conference on Software Engineering*. 3838–3847.
- [19] Pavneet Singh Kochhar, Xin Xia, and David Lo. 2019. Practitioners’ Views on Good Software Testing Practices. In *Proceedings of the 41st International Conference on Software Engineering: Software Engineering in Practice*. 61–70.
- [20] Fang Liu, Yang Liu, Lin Shi, Houkun Huang, Ruifeng Wang, Zhen Yang, and Li Zhang. 2024. Exploring and Evaluating Hallucinations in LLM-Powered Code Generation. *CoRR* abs/2404.00971 (2024). arXiv:2404.00971
- [21] Zhongxin Liu, Kui Liu, Xin Xia, and Xiaohu Yang. 2023. Towards more realistic evaluation for neural test oracle generation. In *Proceedings of the 32nd International Symposium on Software Testing and Analysis*. 589–600.
- [22] Niels Mündler, Mark Niklas Mueller, Jingxuan He, and Martin Vechev. 2024. SWT-Bench Lite Leaderboard. <https://swtbench.com/?results=lite>. Accessed: 2025-09-12.
- [23] Niels Mündler, Mark Niklas Müller, Jingxuan He, and Martin Vechev. 2024. SWT-Bench: Testing and Validating Real-World Bug-Fixes with Code Agents. In *Proceedings of the 38th International Conference on Neural Information Processing Systems*, Vol. 37. 81857–81887.

- [24] Mathieu Nayrolles, Abdelwahab Hamou-Lhadj, Sofiène Tahar, and Alf Larsson. 2015. JCHARMING: A bug reproduction approach using crash traces and directed model checking. In *Proceedings of the 22nd International Conference on Software Analysis, Evolution, and Reengineering*. 101–110.
- [25] Mohammad Masudur Rahman, Foutse Khomh, and Marco Castelluccio. 2020. Why are Some Bugs Non-Reproducible? : –An Empirical Investigation using Data Fusion–. In *Proceedings of the 36th International Conference on Software Maintenance and Evolution*. 605–616.
- [26] Mozhan Soltani, Pouria Derakhshanfar, Annibale Panichella, Xavier Devroey, Andy Zaidman, and Arie van Deursen. 2018. Single-objective versus multi-objectivized optimization for evolutionary crash reproduction. In *Proceedings of the 10th International Symposium on Search-Based Software Engineering*. 325–340.
- [27] Karen Sparck Jones. 1972. A statistical interpretation of term specificity and its application in retrieval. *Journal of documentation* 28, 1 (1972), 11–21.
- [28] Kuo-Chung Tai. 1979. The tree-to-tree correction problem. *J. ACM* 26, 3 (1979), 422–433.
- [29] Dhaval Vyas, Thomas Fritz, and David Shepherd. 2014. Bug Reproduction: A Collaborative Practice Within Software Maintenance Activities. In *Proceedings of the 11th International Conference on the Design of Cooperative Systems*. 189–207.
- [30] Nalin Wadhwa, Atharv Sonwane, Daman Arora, Abhav Mehrotra, Saiteja Utpala, Ramakrishna B Bairi, Aditya Kanade, and Nagarajan Natarajan. 2024. MASAI: Modular Architecture for Software-engineering AI Agents. In *NeurIPS 2024 Workshop on Open-World Agents*.
- [31] Xincheng Wang, Pengfei Gao, Xiangxin Meng, Chao Peng, Ruida Hu, Yun Lin, and Cuiyun Gao. 2025. AEGIS: An Agent-based Framework for Bug Reproduction from Issue Descriptions. In *Proceedings of the 33rd International Conference on the Foundations of Software Engineering*. 331–342.
- [32] Xingyao Wang, Boxuan Li, Yufan Song, Frank F. Xu, Xiangru Tang, Mingchen Zhuge, Jiayi Pan, Yueqi Song, Bowen Li, Jaskirat Singh, Hoang H. Tran, Fuqiang Li, Ren Ma, Mingzhang Zheng, Bill Qian, Yanjun Shao, Niklas Muennighoff, Yizhe Zhang, Binyuan Hui, Junyang Lin, Robert Brennan, Hao Peng, Heng Ji, and Graham Neubig. 2025. OpenHands: An Open Platform for AI Software Developers as Generalist Agents. In *Proceedings of the 13th International Conference on Learning Representations*.
- [33] Chunqiu Steven Xia, Yinlin Deng, Soren Dunn, and Lingming Zhang. 2025. Demystifying llm-based software engineering agents. *Proceedings of the ACM on Software Engineering* 2, FSE (2025), 801–824.
- [34] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. 2024. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. In *Proceedings of the 38th International Conference on Neural Information Processing Systems*, Vol. 37. 50528–50652.
- [35] Zhiqiang Yuan, Mingwei Liu, Shiji Ding, Kaixin Wang, Yixuan Chen, Xin Peng, and Yiling Lou. 2024. Evaluating and Improving ChatGPT for Unit Test Generation. *Proc. ACM Softw. Eng.* 1, FSE, Article 76 (2024), 24 pages.
- [36] Chengming Zhang, Haoye Wang, Chuyang Xu, Jiakun Liu, Kui Liu, and Zhongxin Liu. 2026. Can test cases generated by large language models facilitate automated program repair? *Empirical Software Engineering* 31, 3 (2026), 68.
- [37] Kaizhong Zhang and Dennis Shasha. 1989. Simple Fast Algorithms for the Editing Distance between Trees and Related Problems. *SIAM J. Comput.* 18, 6 (1989), 1245–1262.
- [38] Yuntong Zhang, Haifeng Ruan, Zhiyu Fan, and Abhik Roychoudhury. 2024. AutoCodeRover: Autonomous Program Improvement. In *Proceedings of the 33rd International Symposium on Software Testing and Analysis*. 1592–1604.
- [39] Yuanhe Zhang, Zhiqian Yang, Shengyi Pan, and Zhongxin Liu. 2025. Unit Test Update through LLM-Driven Context Collection and Error-Type-Aware Refinement. In *Proceedings of the 40th International Conference on Automated Software Engineering*. 2504–2515.
- [40] Ziyao Zhang, Chong Wang, Yanlin Wang, Ensheng Shi, Yuchi Ma, Wanjuan Zhong, Jiachi Chen, Mingzhi Mao, and Zibin Zheng. 2025. LLM Hallucinations in Practical Code Generation: Phenomena, Mechanism, and Mitigation. *Proc. ACM Softw. Eng.* 2, ISSTA, Article ISSTA022 (2025), 23 pages.

Our replication package, including the code and data, and our online appendix are available at <https://github.com/ZJU-CTAG/iCoRe>.

A Heuristic Selection

Algorithm 1 provides the full pseudo-code of the heuristic-based selection mechanism described in Section 3.1.

Algorithm 1 Heuristic-based Selection Algorithm for Candidate Nodes

Require: A set of keyword retrieval results $\mathcal{R}$, where each keyword k maps to a set of candidate nodes N_k .

Ensure: The best node $\hat{n}_k$ for each keyword.

```

1: Initialize score for each node  $n$ :  $S(n) \leftarrow 0$ 
2: Initialize score for each file  $f$ :  $F(f) \leftarrow 0$ 
3: for each keyword  $k$  and its corresponding node set  $N_k \in \mathcal{R}$  do
4:   if  $N_k$  is empty then
5:     continue
6:   end if
7:   Base weight  $w_k \leftarrow 1/|N_k|$ 
8:   for each node  $n \in N_k$  do
9:     file  $f \leftarrow$  file containing node  $n$ 
10:     $S(n) \leftarrow S(n) + w_k$  // Uniqueness weight
11:     $F(f) \leftarrow F(f) + w_k$  // File co-occurrence contribution
12:   end for
13: end for
14: for each keyword  $k$  and its corresponding node set  $N_k \in \mathcal{R}$  do
15:   for each node  $n \in N_k$  do
16:     file  $f \leftarrow$  file containing node  $n$ 
17:     parent  $p \leftarrow$  parent node of  $n$  (e.g., class or module)
18:     Calculate total score:  $T(n) \leftarrow S(n) + F(f) + S(p)$  // Weighted sum of three heuristics
19:   end for
20:   Select the node with the highest score from  $N_k$ :  $\hat{n}_k \leftarrow \arg \max_{n \in N_k} T(n)$ 
21: end for
22: return The set of best nodes for all keywords  $\{\hat{n}_k\}$ 

```

B Prompts for iCoRE

Prompt B.1: Prompt for Keywords Extraction

You are a developer investigating a bug report. Your task is to extract critical code elements and relevant technical keywords from the report. These elements should be directly useful for reproducing the bug, or searching the codebase.

Guidelines:

1. Identify and list only the code-related elements that are essential for searching the codebase and understanding or reproducing the bug. These may include function names, class names, method names, variable names, file names, or other identifiers that directly

contribute to debugging.

2. Exclude non-actionable or irrelevant terms, including:

- Generic words like “feature”, “error”, “problem”.
 - User-defined class names or model names that are created within the example bug report but are unlikely to exist in the actual codebase (e.g., A, B, C in a sample model definition).
 - Any abstract or non-code terms that do not directly contribute to debugging.
3. Preserve the exact names or formats of the elements as written in the bug report. If an imported element is renamed using `as`, restore its original module path. For example, if the bug report mentions `import pandas as pd`, and `pd.DataFrame` is used in the code, extract it as `pandas.DataFrame`.
4. Prioritize the extracted elements by their importance for reproducing the bug:
- Elements that are most likely to be useful or necessary for understanding the bug should be ranked highest.
 - Additionally, class names and function/method names should be ranked higher than code fragments.

Output format: Provide the extracted code elements as a list in the following format:

`['ClassName', 'ClassName.methodName', 'functionName', 'variableName']`

For example, if the bug report mentions a class `MyClass` and methods `function1` and `function2` inside it, the output should be: `['MyClass', 'MyClass.function1', 'MyClass.function2']`

Prompt B.2: Prompt for Initial Test Retrieval

You are an assistant for a Python project, specifically helping to write test cases based on a bug report. Your tasks are as follows:

1. Find and recommend some appropriate test functions based on the bug report.
2. Rank the selected test functions in order of relevance, with the most relevant one first.
3. Output the name of test function and its file path. The result should contain at most {topk} test cases.

Available Tools:

- `list_root()`: Lists all files and directories inside the root test folder of the project. You may call this function first.
- `list_folder(path)`: Lists files and directories at the given path.
- `list_classes_and_functions(file_path)`: Lists all classes and functions in a given file.
- `read_function(file_path, function_name)`: Reads the source code of a specific function.

You may need to call `list_folder` multiple times, including on subdirectories, to explore the full directory structure and locate the appropriate test file.

To explore subdirectories, you need to use the relative path (e.g., `tests/folder1/folder2`).

If you find that the initial path you accessed is not suitable, you can access the directory, files, and functions multiple times to find the most appropriate ones.

The maximum number of steps is {max_steps}.

Required Output Format:

Your output must follow this structure:

```
[
  ["path/to/test_file_x.py", "test_function_m"],
  ...
  ["path/to/test_file_y.py", "test_function_n"]
]
```

This list should contain at most {topk} entries, ranked from most to least relevant.

Prompt B.3: Prompt for BRT Generation

You are an assistant responsible for expanding the test suite for our Python project. Your task is to write a test that FAILS on the current buggy code and PASSES after the bug is fixed, without needing any modifications.

Follow these instructions:

1. Understand the Bug Report and explicitly identify the following information: Observed Behavior (OB): What is the actual, incorrect behavior produced by the current code? Expected Behavior (EB): What is the correct behavior that the user expects?

2. Design the Test Logic Based on the Expected Behavior

- Scenario A: If the Expected Behavior (EB) is the “code should run successfully without errors”

Your test case should simply call the problematic code directly.

DO NOT wrap the code in try...except, pytest.raises, or assertRaises.

Reasoning: On the buggy code, the unexpected error (the OB) will be raised, causing the test to FAIL. After the fix, the code will run without error as expected (the EB), causing the test to PASS.

- Scenario B: If the Expected Behavior (EB) is a “specific error or warning should be raised”

Your test case MUST use with pytest.raises(SpecificError): (or an equivalent assertion) to wrap the problematic code.

Reasoning: On the buggy code, the specific error is not raised (the OB), so the assertion will fail, causing the test to FAIL. After the fix, the code raises the correct error as expected (the EB), and the test will PASS.

3. Test Requirement

The test must be minimal and focused: Only reproduce the issue — do not add extra assertions that are unrelated to the bug.

4. Use Provided Information Effectively

The input includes the following sections to assist in test creation:

<issue>: Contains the bug report. Focus on understanding the issue and its context.

<code>: Contains relevant code snippets. These can help you understand the project

structure and usage of specific functions or classes.

Some code snippets may be irrelevant - use only what is necessary for constructing the test case.

If the bug report refers to a class or function that is a user-defined example, do not attempt to import such classes or functions. Instead, define a minimal version of the missing class or function within the test case, just enough to reproduce the bug.

<test>: Contains existing test cases that may relate to the bug. Analyze what each test does, evaluate its relevance to our bug, and determine whether it can be referenced to assist in generating a new test case. The test case you generate will be inserted into the same file as the related tests. Ensure the new test case reproduces the issue and aligns with the overall test suite structure.

Received 2026-02-21; accepted 2026-03-24